

Current best Julia v1.5 SMC²-FC config (RTX 5090, T=28d)

A 2-page technical guide for users and future agents

Generated 2026-05-08

TL;DR — these settings are the bench defaults

The saturated config is now the bench’s default. The minimal invocation is:

```
cd /home/ajay/Repos/python-smc2-filtering-control/version_1_5_Julia
```

```
julia --project=. tools/bench_smc_full_mpc_fsa_gpu.jl \  
    --T-days 28 --seed 42 --output-dir /tmp/v15_T28d_run
```

Wall time on RTX 5090: **19.5 min.** Mean GPU utilisation $\approx 50\%$. VRAM peak ≈ 6.6 GiB (20% of 32 GiB). Output: data.jld2, manifest.json, per_stride.csv, v15_T28d_traces.png, v15_T28d_param_traces.p

If you need to be explicit about every flag (for reproducibility / scripts), the full command is:

```
julia --project=. tools/bench_smc_full_mpc_fsa_gpu.jl \  
    --T-days 28 --replan-K 2 --seed 42 \  
    --N-smc 512          --K-per-chain 1000 \  
    --num-mcmc 3         --max-temp-levels 30 \  
    --ctrl-n-smc 2048    --ctrl-num-mcmc 16 \  
    --ctrl-chees-max 512 --ctrl-n-anchors 12 \  
    --ctrl-max-levels 40 \  
    --output-dir /tmp/v15_T28d_run
```

The two are equivalent at the time of writing.

The framework: SMC2FC_functional

julia/SMC2FC_functional/ is the **default** Julia framework. The previous julia/SMC2FC/ has been re-named to julia/SMC2FC_DEPRECATED_USE_FUNCTIONAL/ and is no longer used by the v1.5 bench. The v1.5 bench’s Project.toml and the model code at version_1_5_Julia/models/fsa_high_res/gpu_pf.jl both import SMC2FC_functional (commits 4a5c007, 9bab259; numerics bit-identical to the deprecated framework on the GPU bench).

Filter-side config (the inner PF)

flag	value	what it controls
--N-smc	512	outer SMC ² posterior particles over θ (10-D filter parameters)
--K-per-chain	1000	inner-PF state particles per chain (latent state cloud size)
--num-mcmc	3	HMC moves per outer tempering level
--max-temp-levels	30	cap on outer tempering levels (typically uses 5–6 in practice)
--ot-max-weight	0.0 (default)	OT rescue OFF — turning it on gives $\sim 12\times$ slowdown
--liu-west-a	0.0 (default)	Liu–West θ -cloud shrinkage OFF (the original bootstrap path)

The filter compute scales roughly as $\mathcal{O}(N_{\text{smc}} \cdot K_{\text{pf}} \cdot T_{\text{steps}})$ per `gpu_log_density` call. At this config, one filter call is sub-millisecond; the filter contributes only $\sim 5\%$ of the bench wall time. Filter VRAM at this config is ~ 300 MB (out of 32 GB available).

Controller-side config (the SMC²-MPC plan finder)

flag	value	what it controls
<code>--ctrl-n-smc</code>	2048	outer SMC ² particles over θ_{ctrl} (RBF coefficients)
<code>--ctrl-num-mcmc</code>	16	ChEES-HMC moves per controller tempering level
<code>--ctrl-chees-max</code>	512	ChEES picker upper bound; candidate list = $[16, 32, 64, \dots, 512]$
<code>--ctrl-n-anchors</code>	12	RBF basis cardinality (controller posterior dimension)
<code>--ctrl-max-levels</code>	40	cap on controller tempering levels (typically uses 5–6)
<code>--ctrl-n-inner</code>	64 (default)	inner CRN-MC trajectories per cost evaluation
<code>--ctrl-target-nats</code>	8.0 (default)	auto-calibration target entropy (nats)
<code>--ctrl-sigma-prior</code>	1.5 (default)	prior std on the RBF coefficient vector
<code>--ctrl-hmc-step</code>	0.2 (default)	base leapfrog step size ε
<code>--ctrl-hmc-leap</code>	16 (default)	base leapfrog count (overridden adaptively by ChEES)

The controller compute dominates wall time: per-replan cost is ~ 40 s, and the bench does ~ 27 replans at $T=28$ d, replan- $K=2$. That is ~ 18 min of the total 19.5 min run.

Why this config (not bigger, not smaller)

Smaller (the historical default) was leaving the GPU empty. The pre-saturation default was $N=32$ $K=200$ `ctrl-n-smc=256` `ctrl-num-mcmc=8` `ctrl-n-anchors=8`: wall 100 s, mean GPU util only 38.5%, the controller settled on a weakly-ramped schedule ($\bar{\Phi} = 0.38$ at stride 21).

Bigger ($N=1024$, $K=4000$, `ctrl-n-smc=4096`, `n-anchors=16`) is wall-time-infeasible. That config did push VRAM to 30.8 GiB (94% saturation) but the JIT-compile plus first-kernel launch on those buffers did not produce log output in 33 min wall time. Future framework engineering (batched OT GPU kernel; kernel-fusion pass) is needed before that tier is practical.

The picked config is the operating point where: (a) the controller has enough compute per replan to converge to the strong “build B then sprint” Banister-overload ramp-up at long horizon, and (b) the whole run finishes in under 30 min wall.

Headline scientific result at this config

metric ($T=28$ d, seed=42, RTX 5090)	default config	this config
total wall time	100 s	1169 s (19.5 min)
mean GPU utilisation	38.5%	49.5%
peak VRAM	6090 MiB	6662 MiB
per-replan controller wall	~ 3 s	~ 40 s ($\sim 14\times$)
applied $\bar{\Phi}$ at stride 21	0.379	0.695 (+83%)
final-stride $\hat{x}_B$ (chronic fitness)	~ 0.105	0.169

The controller now finds the long-horizon Banister-overload ramp-up that the writeup’s §7 hypothesised should emerge at $T=28$ d, with a much stronger commitment than the historical default.

When to deviate from these defaults

- **Want shorter wall time** (e.g. $T=14$ d or $T=7$ d, debug runs): keep `--N-smc 512` `--K-per-chain 1000` but drop `--ctrl-n-smc` to 1024 and `--ctrl-num-mcmc` to 8. Wall time drops $\sim 4\times$ at the cost

of a weaker schedule.

- **Want more VRAM headroom used (memory-bound experiments):** bump `--K-per-chain` to 2000. Should land VRAM around $\sim 10\text{--}12$ GiB without blowing the wall time. Compute scales linearly in K so wall time $\sim 2\times$. Not yet measured at $T=28d$ — run once and check.
- **Need OT rescue** (very low ESS / hard obs models): set `--ot-max-weight 0.01` and accept the $\sim 12\times$ slowdown. The bench will emit a `@warn` reminding you. Out-of-the-box v1.5 does *not* need this — the 3-channel direct-Gaussian obs model is informative enough.
- **Reproducibility:** always set `--seed` explicitly. Default is 42. Keep `--replan-K 2` unless you specifically want more or less frequent re-planning.

Where artefacts live

- **Bench script:** `version_1_5_Julia/tools/bench_smc_full_mpc_fsa_gpu.jl` (this is the live runnable file).
- **Plotting tools:** `compare_v15_julia_vs_python/src/` (the Julia plotting and diagnostic scripts have been consolidated here).
- **Microbench:** `version_1_5_Julia/tools/profile_gpu.jl --both` for filter + controller per-call timings.
- **GPU PF tests:** `version_1_5_Julia/tools/test_gpu_pf.jl`.
- **Lean4 diff test:** `version_1_5_Julia/diff_test/test_lean_diff_v15.jl` (model-math sanity check; should always be green).
- **Saturation findings:** `compare_v15_julia_vs_python/example_run/julia_saturation/FINDINGS_T28d_julia_saturated.md`.
- **Framework:** `julia/SMC2FC_functional/` (the default; *do* import from this). Deprecated framework at `julia/SMC2FC_DEPRECATED_USE_FUNCTIONAL/` (do *not* import from this).